

Intent-Router Cross-Check Gate

Test Suite & Verification Report — Dewlock, the intent-firewall copilot for agentic Sui DeFi

Feature · **Deterministic intent cross-check**

Commit · **91172d7**

Date · **2026-06-26**

Branch · **main**

STATUS · **57/57 PASS**

The moat, applied to intent. The LLM only *proposes* an action; a deterministic router re-derives the action category from the user's literal command and **blocks** any prepared trade whose `actionType` category does not match — catching LLM action-misdetection *before* a PTB ever reaches the user for signing. Same fail-closed posture as the Guardian.

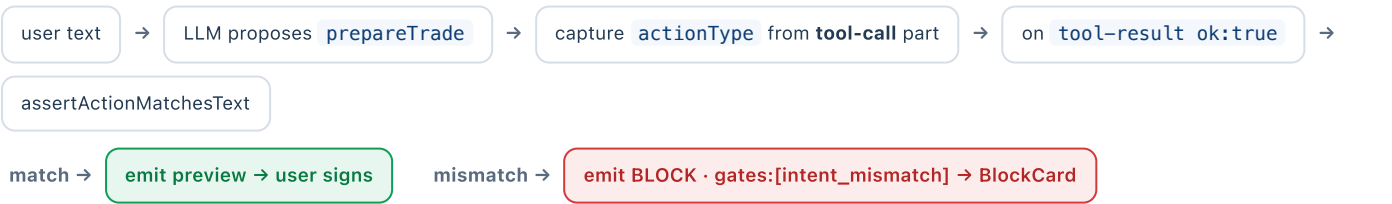
57 AUTOMATED TESTS	57 PASSING	0 FALSE POSITIVES	1103 FULL SUITE (ALL GREEN)	124ms SUITE RUNTIME
------------------------------	----------------------	-----------------------------	---------------------------------------	-------------------------------

1 · What is under test

Module `packages/agent/src/intent/intent-router.ts` (pure, deterministic, no LLM) exposes three units, wired into the agent stream at `apps/web/app/api/agent/route.ts`.

UNIT	RESPONSIBILITY	RETURNS
<code>routeAction(text)</code>	Classify the user's <i>literal</i> command into an action category from adapter keywords (swap / lend / stake / send) + guard-only verbs (bridge / limit). First value-verb of the first clause wins.	category · or <code>null</code> (read-only / no verb)
<code>categoryForActionType(at)</code>	Map a proposed <code>prepareTrade</code> <code>actionType</code> to its category. Composite & DeepBook-lifecycle return <code>null</code> (no single-verb mapping → not cross-checked).	category · or <code>null</code> (skip)
<code>assertActionMatchesText(text, at)</code>	The gate. Blocks only when both sides are non-null and differ. <code>null</code> on either side → pass (no false positive).	<code>{ ok, reason? }</code>

Server wiring — where the gate fires



Why a hard block matters: a misrouted action that reaches the preview is one click from signing the wrong on-chain transaction. The gate replaces the signable preview with a block card carrying the same `{ ok:false, reasons, gates }` shape the Guardian uses — so a misdetected trade can never be signed.

2 · Automated test cases (57)

2.1 · `routeAction` — value-verb classification (21 cases)

#	INPUT (USER TEXT)	EXPECTED	DONE CONDITION
1	swap 5 SUI to USDC	swap	swap keyword detected
2	sell 10 SUI for USDC	swap	<code>sell</code> is a swap synonym
3	lend 5 USDC on navi	lend	lend keyword detected

#	INPUT (USER TEXT)	EXPECTED	DONE CONDITION
4	deposit 100 USDC to suilend	lend	deposit → lend category
5	stake 10 SUI	stake	stake keyword detected
6	stake 5 SUI to afsui	stake	LST provider suffix ignored
7	send 1 SUI to alice	send	send keyword detected
8	transfer 2 USDC to 0xabc	send	transfer → send category
9	pay 1 SUI to bob	send	pay → send category
10	bridge USDC to ethereum	bridge	guard-only verb detected
11	redeem my bridge	bridge	redeem → bridge
12	limit order SUI at 3.50	limit	guard-only verb detected
13	show my portfolio	null	read-only verb → no action
14	what's the best yield	null	question → no action
15	check my balance	null	read-only verb → no action
16	portfolio	null	read-only noun → no action
17	where can I get the best APY?	null	question → no action
18	5 SUI USDC	null	no verb → ambiguous, no action
19	"" (empty)	null	empty → no action
20	" " (whitespace)	null	blank → no action
21	swap 5 SUI to USDC then lend it	swap	first clause wins (chaining handled upstream)

2.2 · `categoryForActionType` — `actionType` → `category` (17 cases)

#	ACTIONTYPE	EXPECTED	DONE CONDITION
22	swap	swap	1:1 map
23	lend_deposit	lend	all lend_* collapse to lend
24	lend_repay	lend	all lend_* collapse to lend
25	lend_borrow	lend	all lend_* collapse to lend
26	lend_withdraw	lend	all lend_* collapse to lend
27	stake	stake	1:1 map
28	unstake	stake	unstake shares stake category
29	transfer	send	transfer → send
30	limit_order	limit	1:1 map
31	bridge_redeem	bridge	1:1 map
32	composite	null · skip	multi-leg chain, no single verb
33	cancel_order	null · skip	DeepBook lifecycle, UI-driven
34	bm_create	null · skip	onboarding wizard, not NL
35	bm_deposit	null · skip	onboarding wizard, not NL
36	withdraw_settled	null · skip	order-management UI
37	claim_settled	null · skip	order-management UI
38	unknown_action	null · skip	unknown → skip (fail-open)

2.3 · `assertActionMatchesText` — the cross-check gate (19 cases)

#	USER TEXT	ACTIONTYPE	RESULT	DONE CONDITION
39	lend 5 USDC on navi	swap	BLOCK	reason names both "swap" & "lend" + "rephrase"
40	stake 10 SUI	lend_deposit	BLOCK	reason names "lend_deposit" & "stake"
41	send 1 SUI to alice	swap	BLOCK	send≠swap → blocked
42	swap 5 SUI to USDC	lend_deposit	BLOCK	swap≠lend → blocked
43	bridge USDC to eth	swap	BLOCK	bridge≠swap → blocked
44	swap 5 SUI to USDC	swap	PASS	category match → allowed
45	lend 5 USDC on navi	lend_deposit	PASS	both → lend
46	deposit 10 USDC to suilend	lend_repay	PASS	both → lend (sub-action differs, OK)
47	stake 10 SUI	stake	PASS	category match
48	stake 10 SUI	unstake	PASS	both → stake category (no over-block)
49	send 1 SUI to alice	transfer	PASS	both → send
50	show my portfolio & 2 more	swap · ...	PASS	read-only text → null → pass (no FP)
51	5 SUI USDC (no verb)	swap	PASS	ambiguous → null → pass (no FP)
52	"" (empty)	swap	PASS	empty → null → pass (no FP)
53	swap... / lend...	composite	SKIP	composite null → pass even if verb present
54	stake 10 SUI	cancel_order	SKIP	DeepBook lifecycle → pass
55	swap 5 SUI	bm_create	SKIP	onboarding → pass
56	stake 10 SUI	withdraw_settled	SKIP	order-mgmt → pass
57	lend 5 USDC on navi	swap	BLOCK	reason is user-safe: no "Error", contains "rephrase"

3 · Manual end-to-end UI test cases

Run against the dev server (`pnpm --filter web dev` → `http://localhost:3000/app`), wallet connected. These exercise the full stream path the unit tests stub out.

M-1 · Block path — forced LLM misdetection

- 1 Type a command whose verb is unambiguous, e.g. `lend 5 USDC on navi` .
- 2 In a misdetection scenario the LLM proposes `prepareTrade(actionType:"swap")` .
- 3 **Expect:** no signable swap preview appears. Instead a **BlockCard** renders with reason *"Refusing for safety: the prepared action (|\"swap|\") doesn't match what you asked (|\"lend|\")..."* and gate `intent_mismatch` .
- 4 **Done when:** nothing signable is shown; user is asked to rephrase; no wallet popup.

M-2 · Pass path — correct detection

- 1 Type `swap 5 SUI to USDC` .
- 2 LLM proposes `prepareTrade(actionType:"swap")` .
- 3 **Expect:** the normal swap preview card renders (Guardian-checked), wallet sign enabled.
- 4 **Done when:** preview shows the correct route & amounts; gate adds no friction to a correct turn.

M-3 · No-false-positive on read-only & chained intents

- 1 Type `show my portfolio` → portfolio renders, gate silent (`routeAction` null).
- 2 Type a composite/atomic intent (`swap 5 SUI to USDC then lend it`) and run atomic → the composite `actionType` is **skipped** by the gate (`categoryForActionType` null); chain proceeds normally.
- 3 **Done when:** neither legitimate flow is blocked.

4 · Verification checklist · done conditions

- ✓ **Unit coverage:** 57/57 automated cases pass (`vitest run intent-router.test.ts` — 124ms).
- ✓ **Full regression:** 1103/1103 tests pass — zero regressions from the gate (1046 prior + 57 new).
- ✓ **Typecheck:** agent + web typecheck clean; the new module emits no errors.
- ✓ **Runtime resolution:** `require("@dewlock/agent/intent/intent-router")` resolves from `apps/web` (where the route runs); package export added & dist built.
- ✓ **Behavior smoke:** `routeAction("lend 5 USDC on navi") = "lend"` ; mismatch vs `swap` returns the safe block reason at runtime.
- ✓ **Block shape:** emitted `{ ok:false, reasons, gates:["intent_mismatch"] }` matches the Guardian block shape → renders via `isPrepareTradeResult` → `makeBlockCard` → `BlockCard` (verified end-to-end).
- ✓ **No false positives:** null on either side passes — read-only, ambiguous, empty, composite, DeepBook all flow through.
- ✓ **Fail-open machinery:** router throw / module-load failure / uncapturable `actionType` → gate skips; only a positive category mismatch is a hard block.
- ✓ **Dev server:** rebuilt agent dist picked up; `/app` returns HTTP 200.
- ✓ **Committed:** 91172d7 on `main` (not pushed).

Known limitation (fail-open, by design): the gate reads `actionType` from the Mastra **tool-call** stream part because the `ok:true` result carries no `actionType` . It relies on the tool-call part arriving before the tool-result in Mastra 1.42's `fullStream` (it does). If that ordering ever changes, the captured `actionType` is null and the gate **fails open** — safe (never blocks a legit turn) but blind. Hardening follow-up: add `actionType` to the `prepareTrade` output schema for a result-side fallback.